Implementação Map-Reduce Concorrente em C++

Ruan Felipe Lauriano Ramos

Departamento de Ciência da Computação – Universidade Federal de Roraima (UFRR)

ruanflauriano@gmail.com

1. Introdução

1.1. O que é Map-Reduce

O Map-Reduce é um modelo de programação para o processamento paralelo de grandes volumes de dados. O paradigma divide a execução em duas fases principais: a fase Map, onde os dados de entrada são particionados e processados de forma independente por múltiplos workers gerando resultados parciais, e a fase Reduce, que consolida esses resultados parciais em um único resultado final.

1.2. Objetivo

Desenvolver e validar uma solução concorrente em C++ utilizando a biblioteca POSIX pthread. O projeto explora particionamento de dados, sincronização por barreira fork-join e abstração de operações via ponteiros de função, com foco em escalabilidade, análise de speedup e ausência de anomalias de concorrência.

2. Desenvolvimento

2.1. Arquitetura

A solução implementa um pipeline paralelo determinístico em três fases:

a. **Particionamento e MAP:** O vetor de N elementos é dividido em T chunks. O tamanho base é $\lfloor N/T \rfloor$, com o resíduo (N % T) absorvido pela última thread. Cada thread recebe um ponteiro exclusivo para seu sub-vetor encapsulado na estrutura ThreadArgs, garantindo isolamento total de memória — cada thread lê e escreve apenas em sua própria região, sem nenhuma variável compartilhada na fase MAP.
b. **Barreira de Sincronização:** Controle fork-join via pthread_join. A thread principal suspende a execução até que todos os workers concluam a fase MAP, garantindo que os resultados parciais estejam disponíveis antes da redução.
c. **REDUCE:** Operação serial executada pela thread principal. A função reduce_sum itera sobre o vetor de ThreadArgs acumulando os resultados parciais sobre um valor de identidade neutro (0 para soma, LLONG_MIN para máximo).

2.2. Protótipo Inicial (Teste_MapReduce.cpp)

A versão inicial estabeleceu a mecânica básica do algoritmo com N e T fixos em código (hardcoded) e operação de soma como única função de map. Validou o uso de pthreads para subdivisão de tarefas, porém sem flexibilidade de configuração em tempo de execução.

2.3. Algoritmo Final (Algoritmo_MapReduce.cpp)

O protótipo evoluiu para um framework configurável. As principais melhorias foram:

- **Ponteiros de função genéricos:** os tipos MapFn e ReduceFn permitem substituir a operação sem alterar o motor de concorrência — a estrutura ThreadArgs carrega a função a ser aplicada junto com os dados, desacoplando lógica de negócio da infraestrutura paralela.
- **Auto-detecção de hardware:** sysconf(_SC_NPROCESSORS_ONLN) no Linux e GetSystemInfo no Windows retornam o número de threads lógicas disponíveis, usado como padrão quando -t não é especificado.
- **Interface CLI:** flags -n, -t, -auto e -bench permitem configurar o algoritmo em tempo de execução sem recompilar.
- **Módulo de benchmark:** mede o tempo médio de 5 execuções para T = 1 até hw e calcula o speedup relativo ao caso single-thread.

2.4. Desafios de Concorrência

a. **Race Conditions:** Eliminadas por design. Como cada ThreadArgs aponta para uma região distinta do vetor (sem sobreposição), não há acesso simultâneo à mesma posição de memória durante a fase MAP. Isso torna mutexes desnecessários — o isolamento é estrutural, não sincronizado.
b. **Deadlock:** Impossível pela topologia do algoritmo. O padrão fork-join é unidirecional: a thread principal lança os workers e aguarda todos terminarem antes de prosseguir. Não há espera circular nem aquisição de múltiplos recursos, que são as condições necessárias para deadlock.
c. **Overhead de criação de threads:** Para vetores pequenos, o custo de criar e destruir threads pode superar o ganho do paralelismo. O módulo de benchmark quantifica esse fenômeno — o speedup real com N baixo frequentemente fica abaixo do ideal teórico, especialmente para T elevado.

2.5. Validação

A correção dos resultados concorrentes é verificada por fórmulas analíticas de tempo O(1), independentes da implementação testada. Isso garante que um bug nos chunks ou na redução seja detectado imediatamente:

- **Soma aritmética:** comparada com $S = N \cdot (N+1)/2$ (progressão de Gauss). Detecta elementos duplicados ou omitidos nos chunks.
- **Máximo:** comparado com M = N, pois o vetor contém os inteiros 1..N. Detecta erros no reduce_max.
- **Contagem de pares:** comparada com $C = \lfloor N/2 \rfloor$. Detecta erros de particionamento onde elementos são contados em mais de um chunk.

Cada teste imprime o resultado obtido, o valor esperado e o status [OK] ou [ERRO].

2.6. Especificações de CLI

A interface de linha de comando expõe os seguintes controles:

Flag	Funcionalidade	Padrão
-n <N>	Define o tamanho do vetor	1.000.000
-t <T>	Define o número de threads	Auto (hardware)
-auto	Usa todas as threads lógicas da máquina	Ativado por padrão
-bench	Executa benchmark de speedup T=1..hw	Desativado

2.7. Testes

Testes de Casos de Borda

- Vetor Vazio (N=0):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -n 0
Threads invalido.
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

- Vetor Unitário (N=1):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -n 1 -t 1
N=1  T=1  (hardware: 12 threads)

Operacao        Resultado      Esperado       Status
-------------------------------------------------------
Soma total      1              1              [OK]
Maximo          1              1              [OK]
Contar pares    0              0              [OK]
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

- Threads superiores aos elementos (T > N):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -n 5 -t 10
Threads invalido.
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

- Divisão inexata (N=10, T=3):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -n 10 -t 3
N=10  T=3  (hardware: 12 threads)

Operacao         Resultado      Esperado       Status
-----------------------------------------------------
Soma total       55             55             [OK]
Maximo           10             10             [OK]
Contar pares     5              5              [OK]
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

Testes de Performance e Benchmark

- Benchmark padrão (1 até threads de hardware):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -bench
N=1000000  T=12  (hardware: 12 threads)

Operacao         Resultado      Esperado       Status
-----------------------------------------------------
Soma total       500000500000   500000500000   [OK]
Maximo           1000000        1000000        [OK]
Contar pares     500000         500000         [OK]

Benchmark (soma, N=1000000)
Threads    Tempo(ms)    Speedup
--------------------------------
1          1.079        1.00x
2          0.866        1.25x
3          0.619        1.74x
4          0.641        1.68x
5          0.725        1.49x
6          0.619        1.74x
7          0.884        1.22x
8          0.864        1.25x
9          0.875        1.23x
10         0.640        1.69x
11         0.953        1.13x
12         0.656        1.64x
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

- Benchmark com carga elevada (N=100M):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -n 100000000 -bench
N=100000000  T=12  (hardware: 12 threads)

Operacao         Resultado      Esperado       Status
-----------------------------------------------------
Soma total       5000000050000000 5000000050000000 [OK]
Maximo           100000000      100000000      [OK]
Contar pares     50000000       50000000       [OK]

Benchmark (soma, N=100000000)
Threads    Tempo(ms)    Speedup
--------------------------------
1          35.752       1.00x
2          20.468       1.75x
3          14.898       2.40x
4          13.120       2.73x
5          15.834       2.26x
6          17.743       2.01x
7          13.161       2.72x
8          11.133       3.21x
9          11.281       3.17x
10         11.735       3.05x
11         10.434       3.43x
12         10.345       3.46x
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

- **Execução automática (todas as threads disponíveis):**

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -auto
N=1000000  T=12  (hardware: 12 threads)

Operacao         Resultado      Esperado       Status
-----------------------------------------------------
Soma total       500000500000   500000500000   [OK]
Maximo           1000000        1000000        [OK]
Contar pares     500000         500000         [OK]
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

Validação de Integridade

- Single-thread (Baseline):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -t 1
N=1000000  T=1  (hardware: 12 threads)

Operacao         Resultado      Esperado       Status
-----------------------------------------------------
Soma total       500000500000   500000500000   [OK]
Maximo           1000000        1000000        [OK]
Contar pares     500000         500000         [OK]
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

- Carga Customizada (N=5M, T=8):

```
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$ ./map_reduce -n 5000000 -t 8
N=5000000  T=8  (hardware: 12 threads)

Operacao         Resultado      Esperado       Status
-----------------------------------------------------
Soma total       12500002500000 12500002500000 [OK]
Maximo           5000000        5000000        [OK]
Contar pares     2500000        2500000        [OK]
ruanflau@LauriComputador:/media/ruanflau/Storage/Projetos/Avaliação I SO/Algoritmo Map Reduce$
```

3. Conclusão

A implementação do framework Map-Reduce em C++ com pthreads atingiu os objetivos propostos em corretude, robustez e escalabilidade. O particionamento por aritmética de ponteiros garantiu isolamento total de memória na fase MAP, eliminando a necessidade de mutexes e evitando race conditions por design. O padrão fork-join via ***pthread_join*** tornou deadlock estruturalmente impossível.

A validação analítica por fórmulas fechadas O(1) confirmou integridade em todas as operações e configurações testadas, incluindo casos de borda como vetor unitário, divisão inexata e T > N. Os testes de benchmark revelaram o comportamento previsto pela Lei de Amdahl: para N = 1M o speedup estagna em ~1.7x devido ao overhead de criação de threads, enquanto para N = 100M o ganho paralelo supera esse custo e atinge 3.46x com 12 threads — demonstrando empiricamente que o paralelismo de granularidade fina só é vantajoso quando o volume de trabalho por thread justifica o custo de sincronização.